PRACTICAL – 10

Implement Kruskal's algorithm

Aim:
To implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.

Objective:
To implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph by selecting edges in increasing order of weight without forming cycles.

Algorithm:

- Sortall edges in ascending order of their weights.
- Select the edge with the minimum weight.
- Check whether adding the edge creates a cycle.
- If no cycle is formed, add the edge to the MST.
- Repeat until \(V-1\) edges are selected.
- Calculate and display the total weight of the MST.

Code:

```
V = 5

edges = [
    (0, 1, 2),
    (0, 3, 6),
    (1, 2, 3),
    (1, 3, 8),
    (1, 4, 5),
    (2, 4, 7),
    (3, 4, 9)
]

edges.sort(key=lambda x: x[2])

parent = list(range(V))
rank = [0] * V
```

```
def find_parent(i):
    if parent[i] != i:
        parent[i] = find_parent(parent[i])
    return parent[i]

def union_set(x, y):
    root_x = find_parent(x)
    root_y = find_parent(y)

    if rank[root_x] < rank[root_y]:
        parent[root_x] = root_y
    elif rank[root_x] > rank[root_y]:
        parent[root_y] = root_x
    else:
        parent[root_y] = root_x
        rank[root_x] += 1

total = 0
count = 0

print("Edges in MST:")

for u, v, weight in edges:
    root_u = find_parent(u)
    root_v = find_parent(v)

    if root_u != root_v:
        print(u, "-", v, ":", weight)
        total += weight
        union_set(root_u, root_v)
        count += 1

    if count == V - 1:
        break

print("Total weight of MST =", total)
print("EN.No: 92460118698")
```

Output:

```
PS C:\Users\manvi\OneDrive\Desktop\daa practical 10> & C:\Users\manvi\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/manvi/OneDri
ve/Desktop/daa practical 10/Kruskal_Algorithm.py"
Edges in MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Total weight of MST = 16
EN.No: 92460118698
PS C:\Users\manvi\OneDrive\Desktop\daa practical 10>
```

Time Complexity:

Operation	*Time Complexity*	*Space Complexity*
Sorting edges	*O(E log E)*	O(1) auxiliary
Find and Union	*O(E alpha(V))*	O(V)
Overall	*O(E log E)*	O(V) auxiliary